

Project Report: DocuLingua

GROUPE MEMEBERS: Amirarsalan Sanati, Diptaraj Sen

1. Project Idea

DocuLingua is a Python and Streamlit application that helps language learners study a foreign-language document by turning uploaded PDF or TXT content into a structured learning guide. The current repository focuses on French source documents with English explanations, and the intended users are learners who want vocabulary, grammar, phrases, exercises, and review material from real texts rather than generic textbook examples.

The application extracts and cleans document text locally, prepares document statistics and chunks, generates learning sections through configurable LLM providers, and exports a static PDF workbook. The repository uses Streamlit for the user interface, PyMuPDF for PDF text extraction and fallback PDF generation, optional WeasyPrint rendering, Pydantic models for guide structure, Groq and Gemini provider adapters for LLM output, python-dotenv and pydantic-settings for configuration, and pytest for automated tests.

2. Achieved Tasks and Features

- Supports document input through Streamlit file upload and previously stored uploads.
- Accepts .pdf and .txt files and rejects unsupported file extensions.
- Extracts PDF text with PyMuPDF and reads TXT files as UTF-8 text.
- Cleans text, chunks content, and calculates document statistics such as characters, words, paragraphs, unique words, and estimated reading time.
- Provides a Streamlit preview workflow with metrics, cleaned text preview, and first chunk preview.
- Generates learning-guide sections for overview, key vocabulary, important verbs, grammar patterns, useful phrases, mini lessons, practice exercises, reading practice, review sheet, and answer key.
- Uses a provider router that tries configured LLM providers in order, with Groq as the primary provider and Gemini as fallback by default.
- Parses JSON responses, validates section data with Pydantic-backed structures, records provider/model metadata, and reports failed sections.
- Includes optional final guide polishing through additional LLM calls.

- Renders a downloadable static PDF guide using WeasyPrint when available and PyMuPDF as a fallback.
- Stores uploads and generated PDFs locally under `app/storage/`.
- Provides API key settings in the Streamlit interface and supports `.env` configuration.
- Includes automated pytest coverage for document loading, text cleaning, chunking, provider routing, response parsing, guide generation, PDF building, progress tracking, and schemas.

3. Software Development Principles Followed

- **Modularity:** The project separates UI, core processing, learning content generation, LLM providers, PDF rendering, and utilities into distinct modules.
- **Separation of concerns:** `streamlit_app.py` handles interaction, `app/core/` handles document processing, `app/learning/` handles guide content, `app/llm/` handles prompts and providers, and `app/pdf/` handles PDF output.
- **Design modeling:** Created design models (usecase, sequence, architecture etc.) following design modeling principles.
- **Complying with principles guiding process:** Create something which brings value for the user.
- **Single responsibility:** Individual helpers such as document loaders, text cleaners, response parsers, provider adapters, and PDF builders each focus on one main responsibility.
- **Configuration management:** API keys, model names, provider order, and project settings are loaded through `.env` and `pydantic-settings`.
- **Input validation:** Supported file types are checked, TXT decoding errors are handled, and LLM output is validated before being accepted into the guide.
- **Error handling and fallback:** The provider router records failed attempts, retries invalid JSON, falls back from Groq to Gemini, and the PDF builder falls back from WeasyPrint to PyMuPDF.
- **Extensibility:** New LLM providers can be added by implementing the provider interface and registering them with the router.
- **Testing:** The `tests/` directory contains focused pytest cases that cover local processing and mocked provider behavior without requiring real API calls.
- **Continuous Integration:** CI practices were involved during the development of the software: Regular and granular commits (pushes), unit testing, fall back, github workflows, github actions etc.
- **Documentation:** The README explains the project goal, setup, environment variables, run command, provider design, and current MVP status.

4. Demo and Repository Information

- Repository link: <https://github.com/diptaraj23/DocuLingua>

- Run command:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
streamlit run streamlit_app.py
```

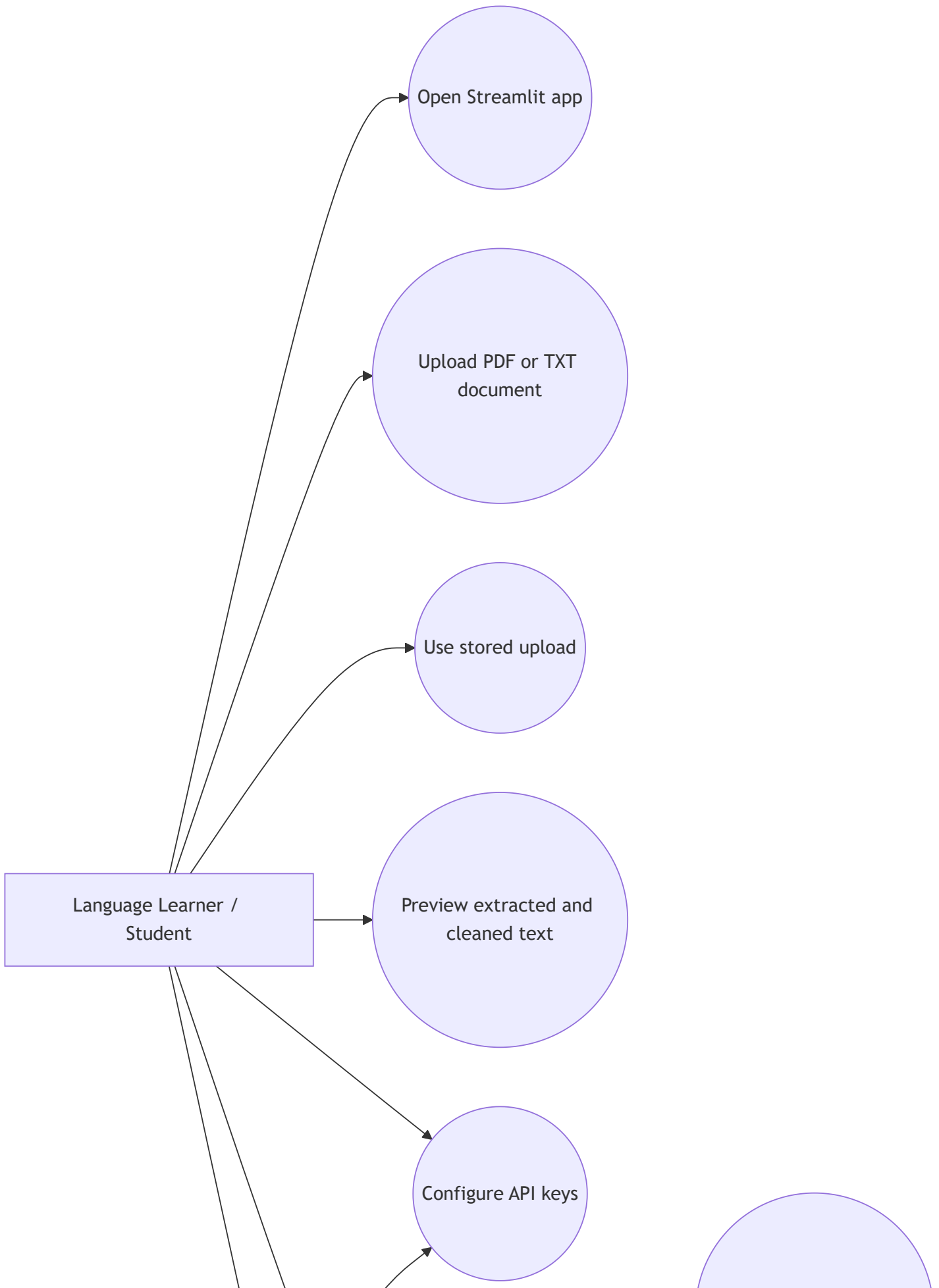
- Required dependencies are listed in `requirements.txt` : Streamlit, Groq, Google GenAI, python-dotenv, Pydantic, pydantic-settings, PyMuPDF, spaCy, pytest, and WeasyPrint.
- Required environment variables for LLM generation include `GROQ_API_KEY` and optionally `GEMINI_API_KEY` . Model and provider settings are also configurable in `.env` .
- LLM generation requires at least one configured API key. The PDF renderer can use PyMuPDF if WeasyPrint native dependencies are unavailable.

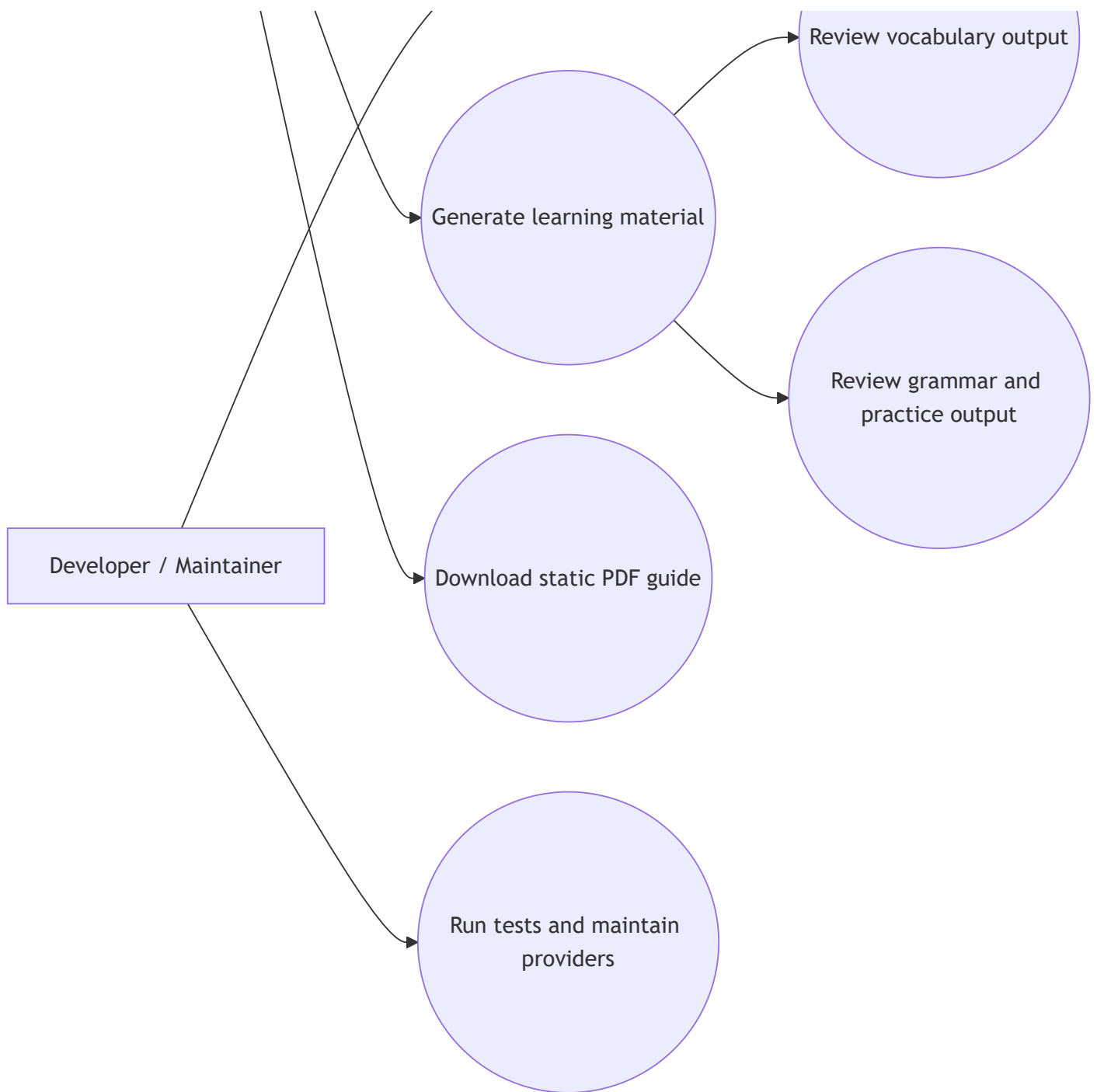
5. Annexes

In this part, we introduce and explain different diagrams illustrating the design of the system.

Annex A: UML and architecture diagrams.

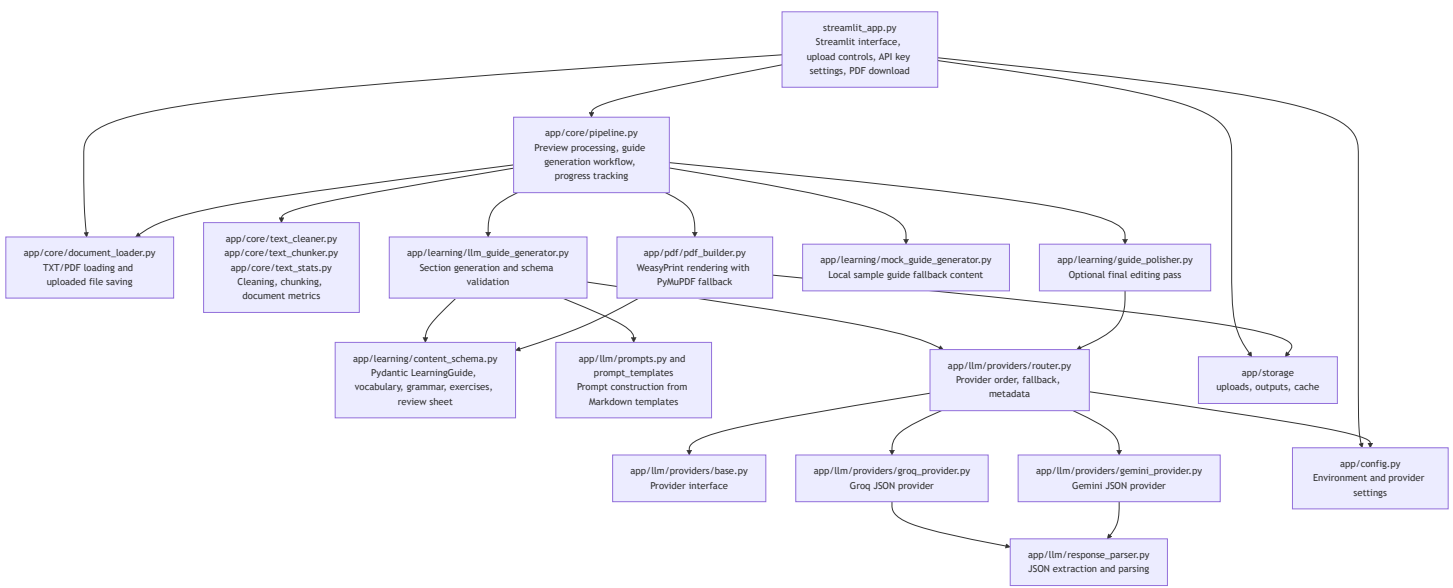
Use Case Diagram





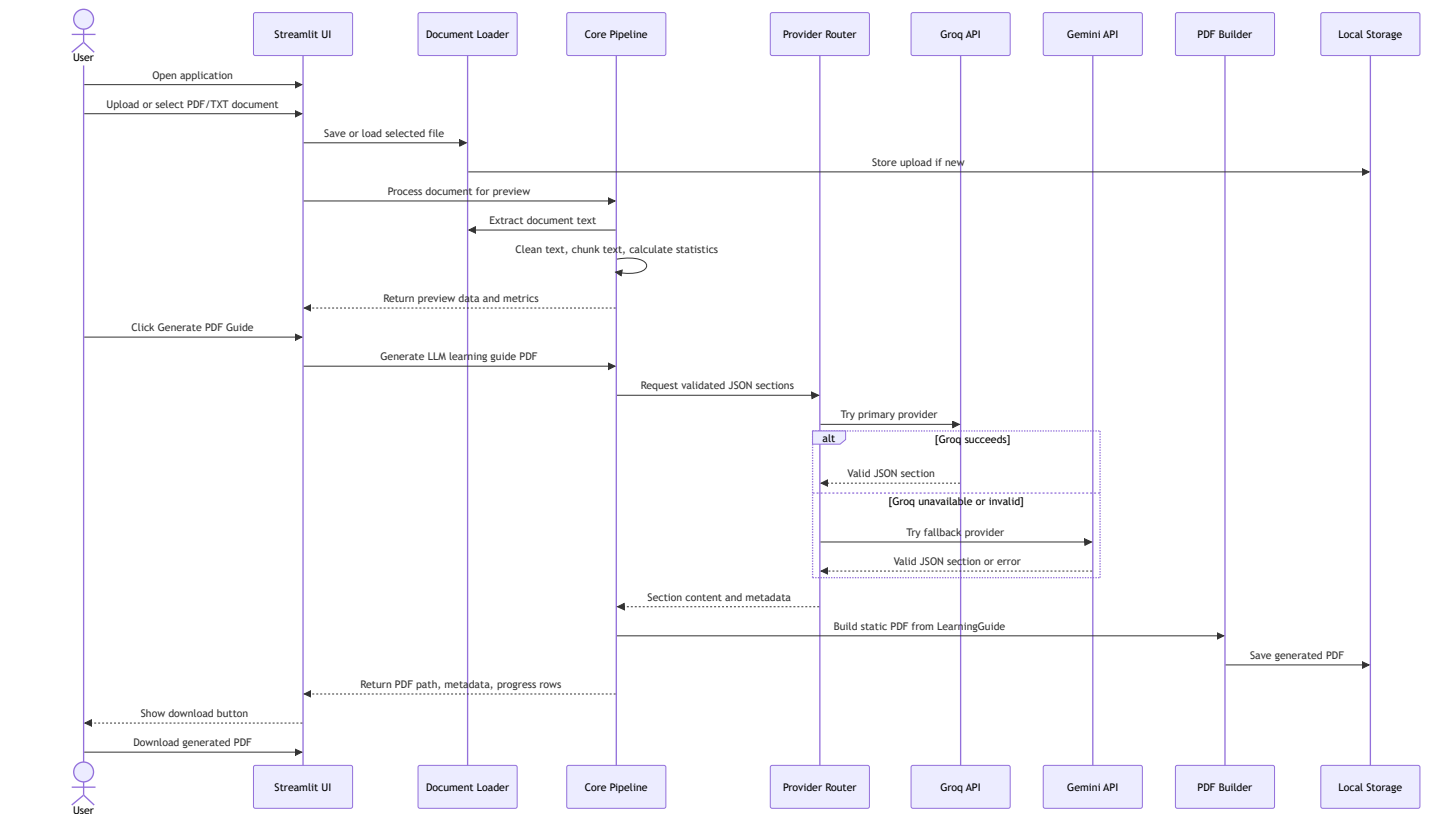
This diagram shows the main learner actions: opening the Streamlit app, uploading or selecting a document, previewing extracted text, configuring API keys, generating learning material, and downloading the final PDF. It also includes developer maintenance actions such as running tests and managing providers.

Class / Module Diagram



This diagram represents the repository as cooperating modules. The Streamlit UI calls the core pipeline, the pipeline uses document loading and text processing helpers, the learning layer builds Pydantic-based guide structures, the provider router manages Groq and Gemini adapters, and the PDF builder writes the final workbook.

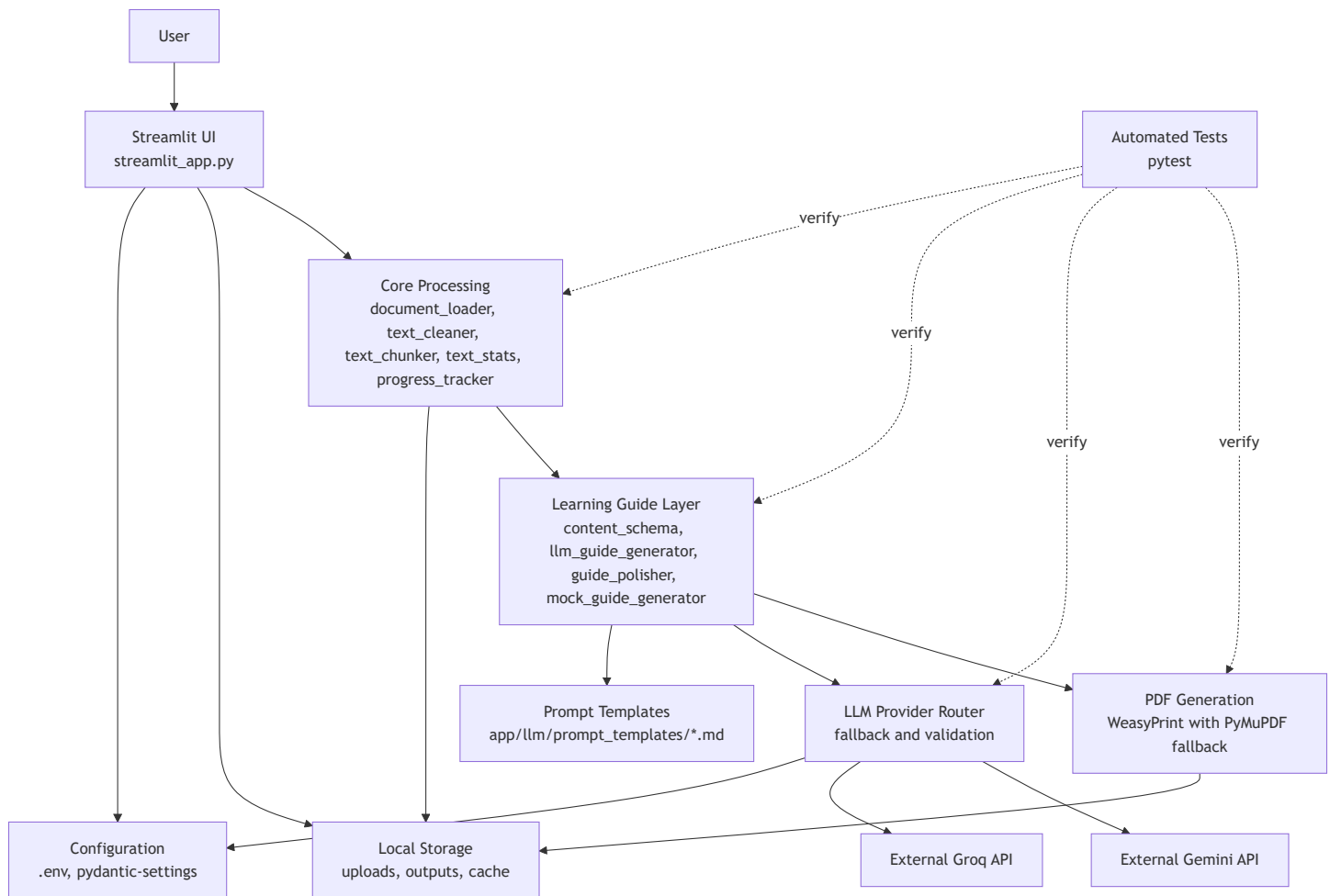
Sequence Diagram



This diagram follows the main application workflow from opening DocuLingua through document upload, local extraction, preview generation, LLM section generation, static PDF creation, and final

download.

Architecture Diagram



This diagram summarizes the runtime architecture: Streamlit UI, configuration, local storage, core processing, learning-guide generation, prompt templates, provider routing, external LLM APIs, PDF generation, and pytest coverage.

Database / Storage

The inspected repository does not contain a database schema, ORM configuration, migration files, or database service configuration. DocuLingua stores files locally instead:

- Uploaded source documents are stored under `app/storage/uploads/`.
- Generated PDF learning guides are stored under `app/storage/outputs/`.
- Cache-related placeholder storage exists under `app/storage/cache/`.
- Runtime configuration is stored in `.env`.

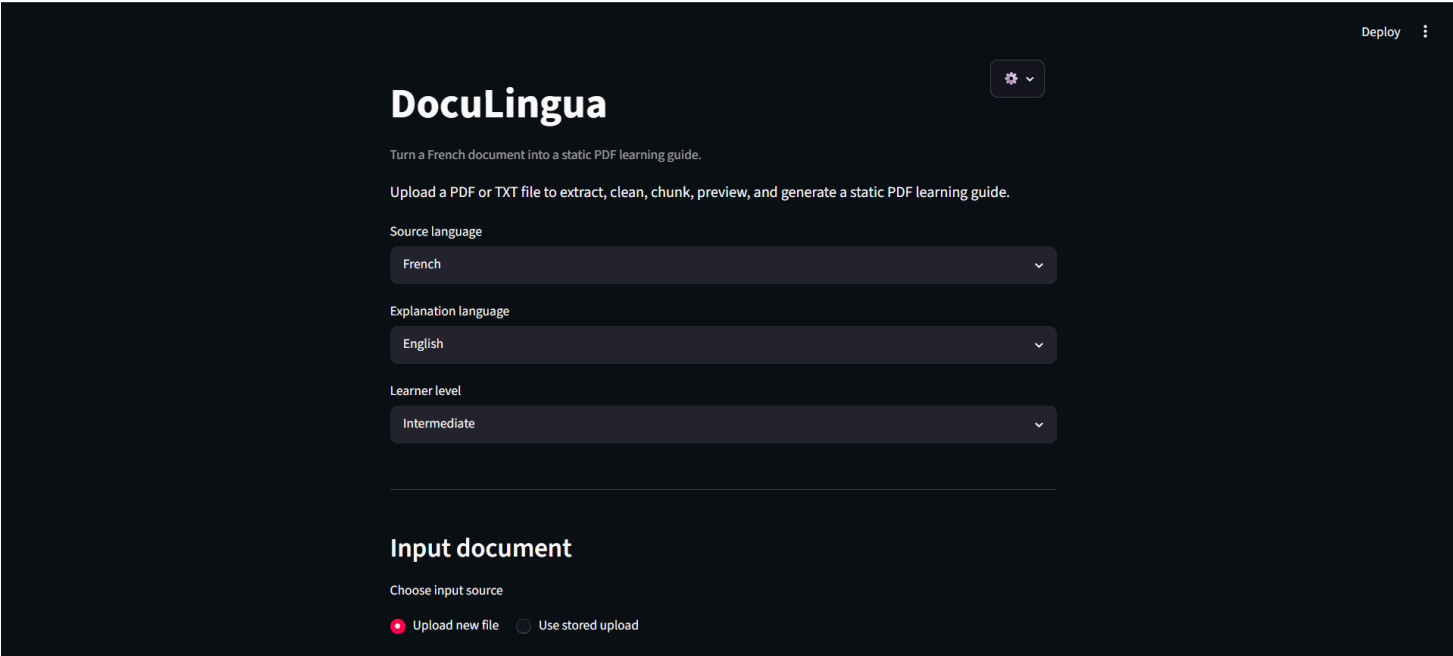
Because the MVP does not use a database, an ER diagram is not applicable. A local storage view is included in the architecture diagram.

The project does not use a database. Local file storage is used for uploads, generated PDFs, cache placeholders, and environment configuration.

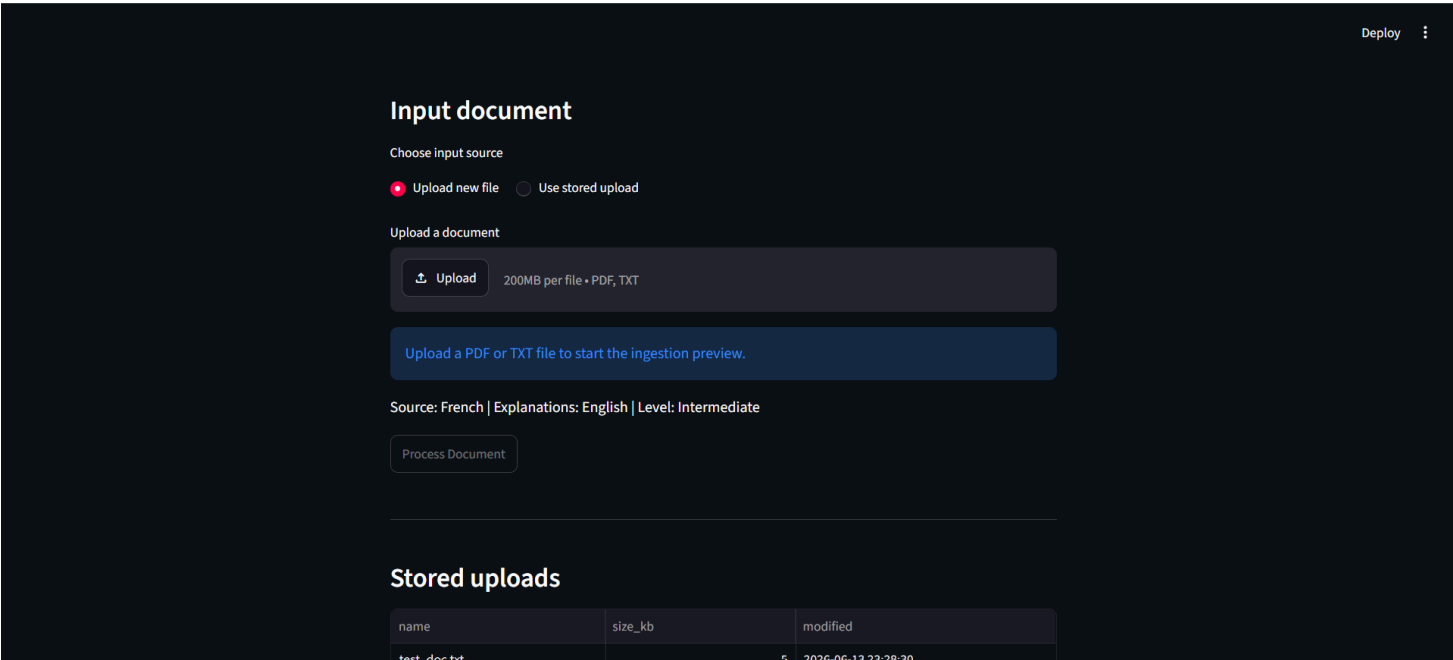
Annex B: Screenshot placeholders.

In this section, we will showcase some screenshots from the software itself, to better demonstrate the developed product and its capabilities:

Main page, main functionalities.



Here, is the user input interface:



There's an option for the user to select previously uploaded docs (for regeneration):

Deploy

Input document

Choose input source

Upload new file

Use stored upload

Choose a previously uploaded file

test_doc.txt

Ready to process stored file: test_doc.txt

Source: French | Explanations: English | Level: Intermediate

Process Document

Stored uploads

name	size_kb	modified
test_doc.txt	5	2026-06-13 23:28:30

And you can also choose to delete your previously uploaded documents:

Deploy

Stored uploads

name	size_kb	modified
test_doc.txt	5	2026-06-13 23:28:30

test_doc.txt5 KBDelete

PDF Learning Guide

Generate a static PDF guide from the uploaded document. LLM providers generate the learning sections. The guide does not create sentence-wise translation or interactive exercises.

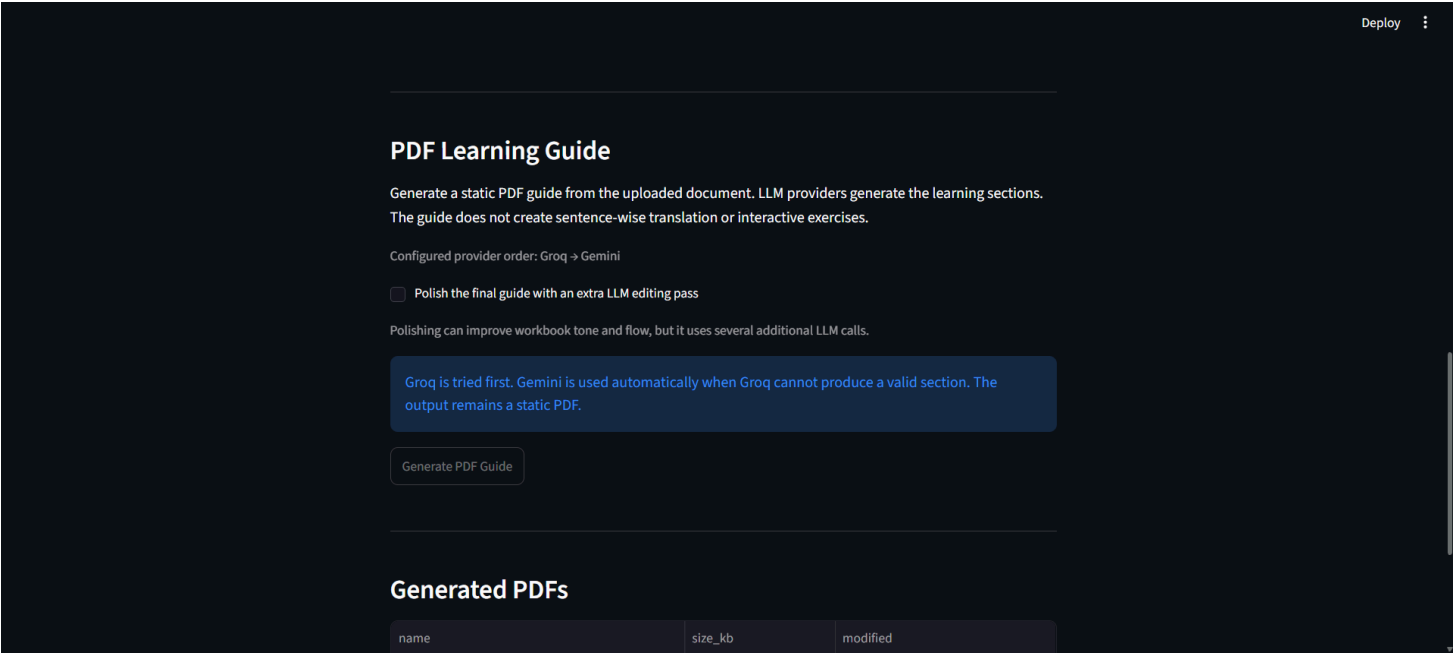
Configured provider order: Groq → Gemini

Polish the final guide with an extra LLM editing pass

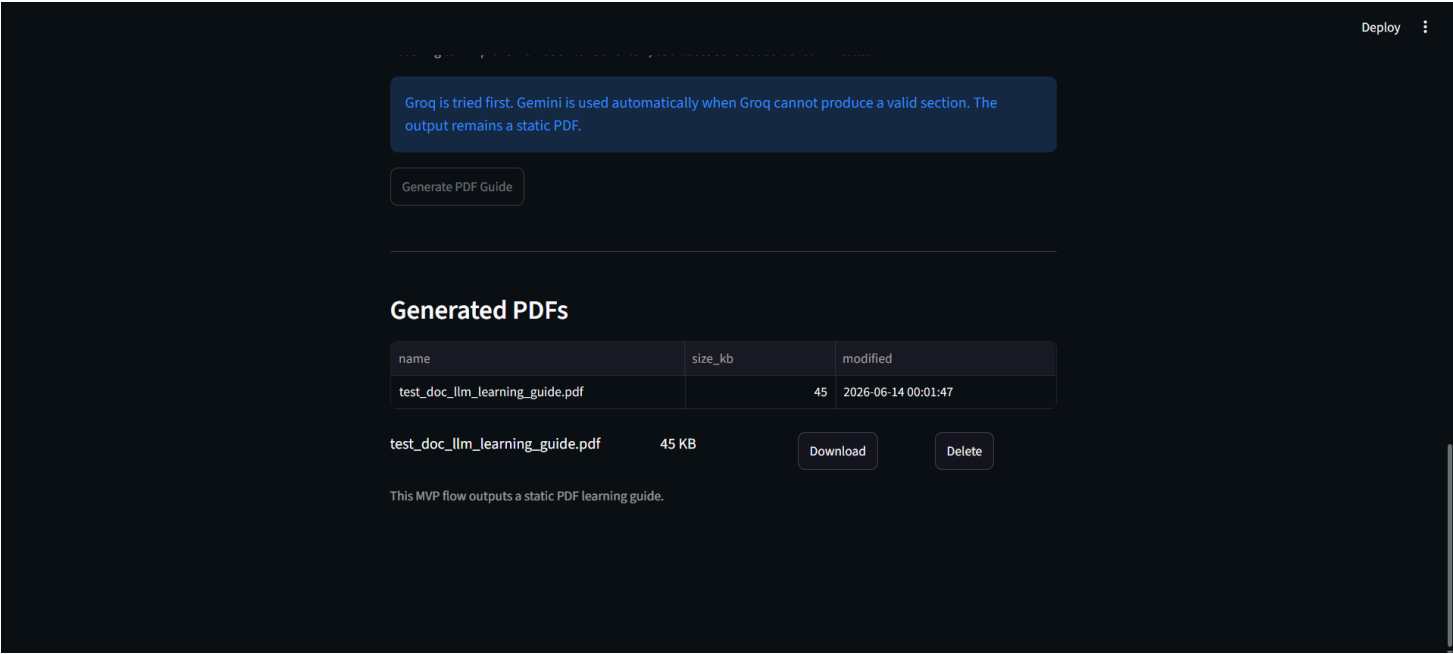
Polishing can improve workbook tone and flow, but it uses several additional LLM calls.

Groq is tried first. Gemini is used automatically when Groq cannot produce a valid section. The output remains a static PDF.

Here, is the learning guide pdf generation user interface:

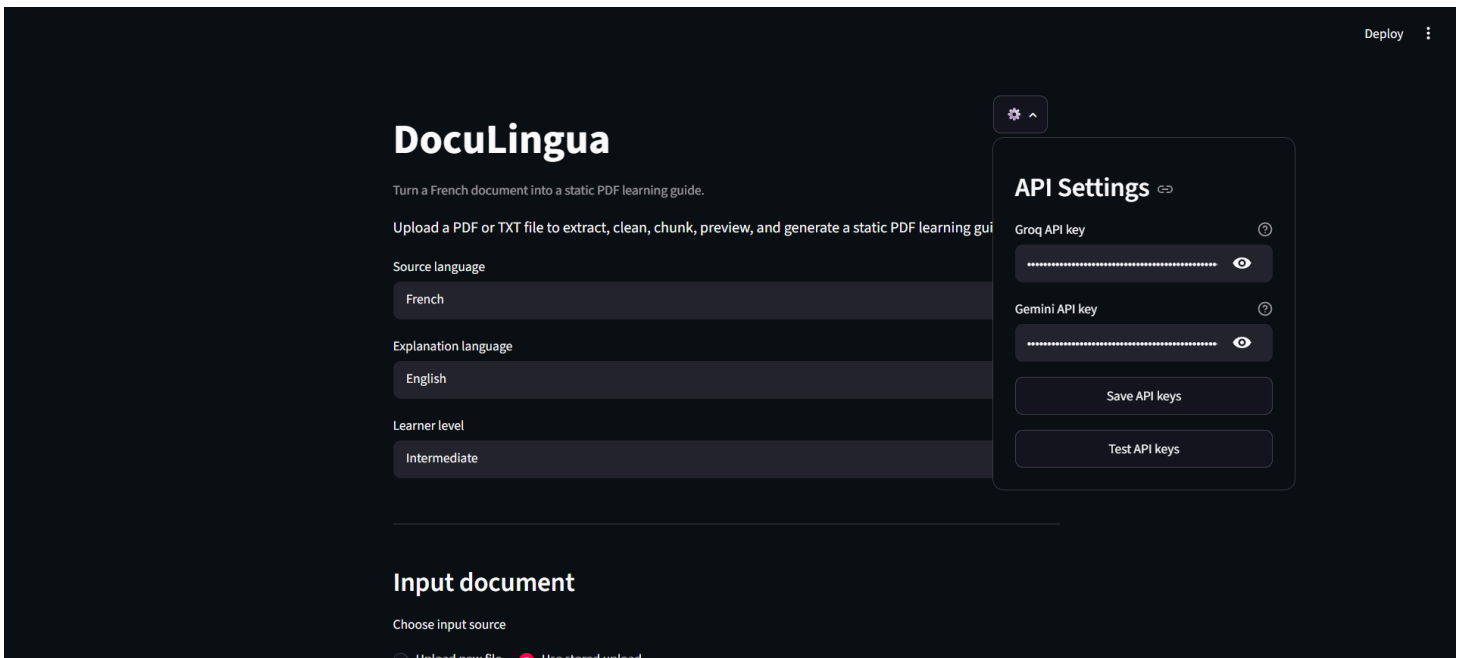


And of course you can see and manage previously generated materials:

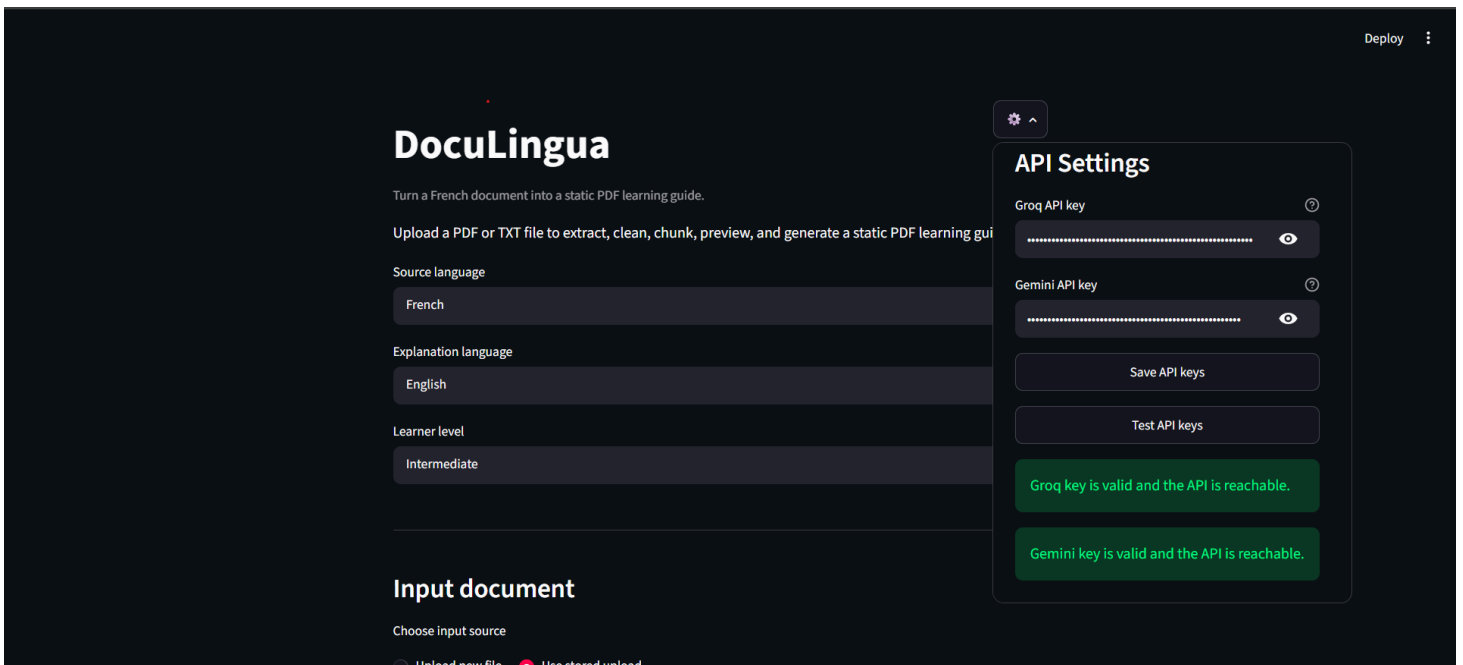


API Settings.

User can change their api keys without having to directly edit '.env' file:

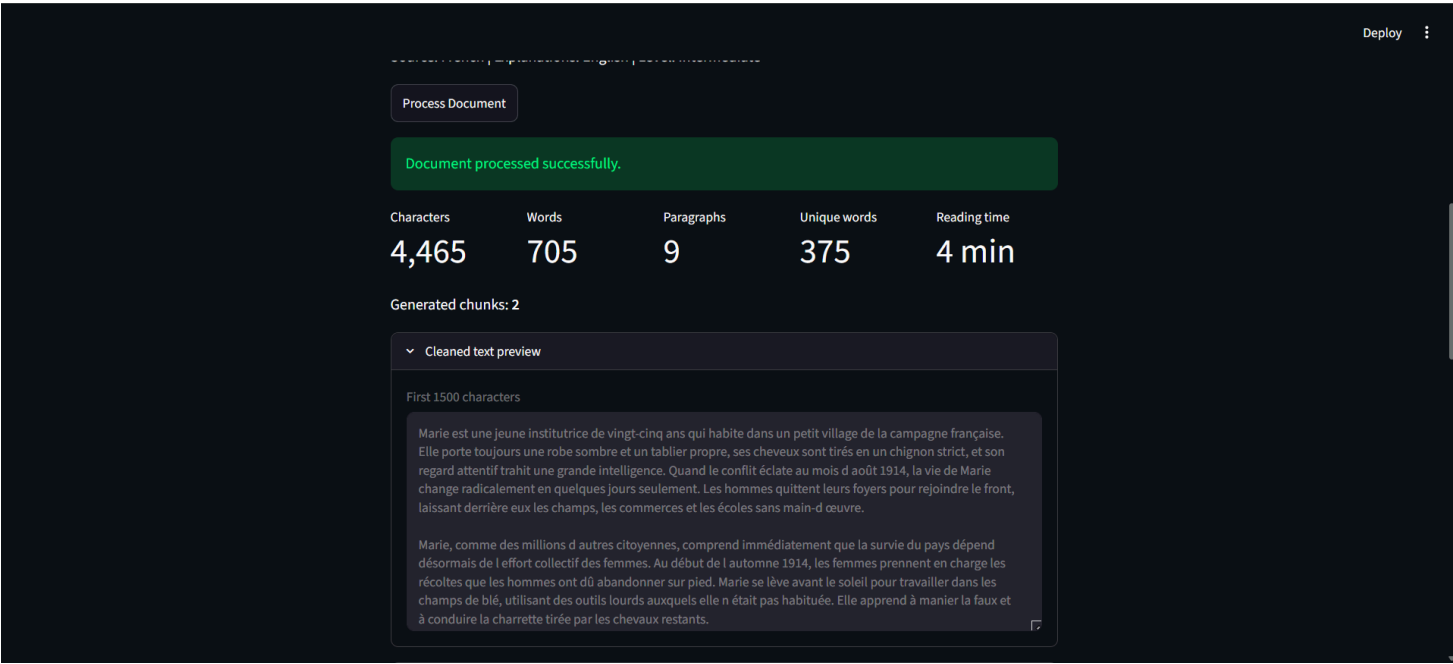


User can save, or test the connectivity and authenticity and responsiveness of the keys:

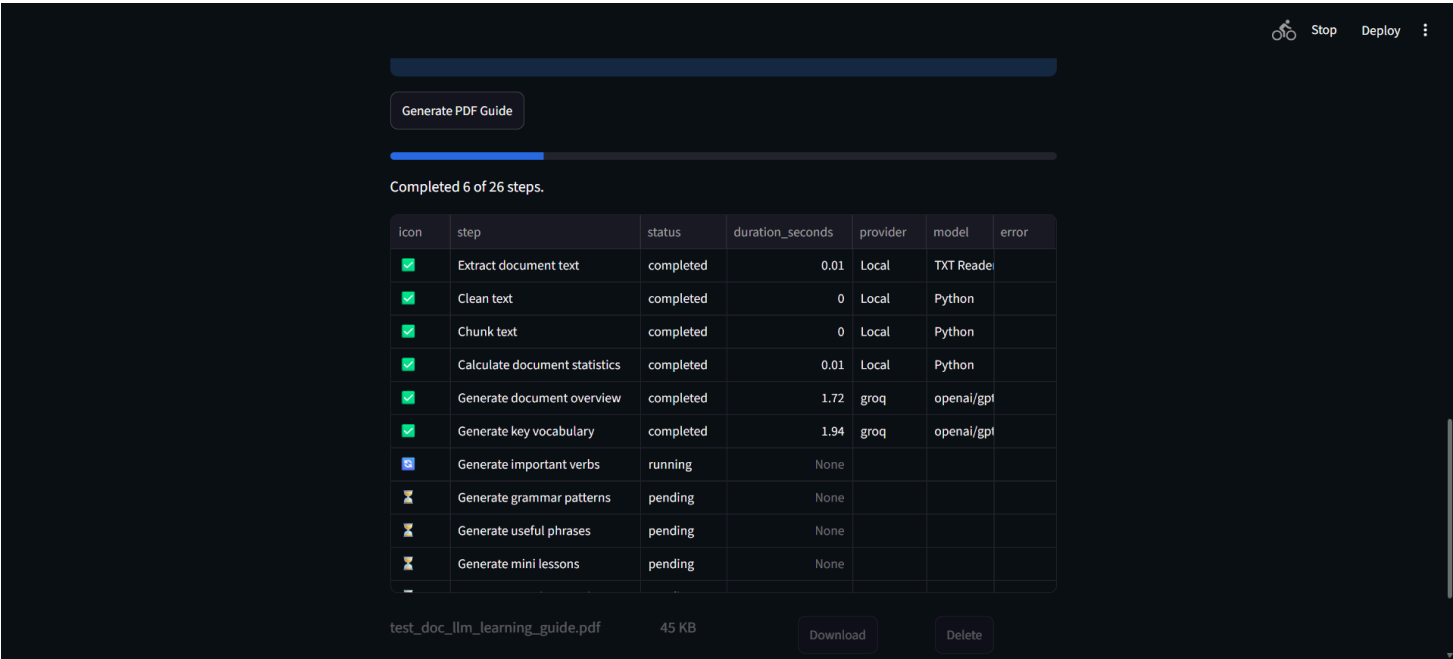


Sample run.

After uploading document, the uploaded material will be processed and user will be given some stats regarding the doc:



User can see the learning guide generation progress, its pipeline, and its current stage, along with the time took to complete the task and the model used by software to complete it.



After generation is complete, user can download the generated document. Moreover, generation report will be shown to the user:

Total processing time: 296.21 seconds

Final polishing enabled: yes

Polishing succeeded: yes

section	provider	model	success
Polish document overview	groq	openai/gpt-oss-120b	
Polish key vocabulary	groq	openai/gpt-oss-120b	
Polish topic vocabulary groups	groq	openai/gpt-oss-120b	
Polish important verbs	groq	openai/gpt-oss-120b	
Polish grammar patterns	groq	openai/gpt-oss-120b	
Polish useful phrases	groq	openai/gpt-oss-120b	
Polish mini lessons	groq	openai/gpt-oss-120b	
Polish practice exercises	groq	openai/gpt-oss-120b	
Polish reading practice	groq	openai/gpt-oss-120b	
Polish review sheet	groq	openai/gpt-oss-120b	

LLM-generated sections: Document Context Overview, Key Vocabulary, Important Verbs, Grammar Patterns, Useful Phrases and Expressions, Mini Language Lessons, Practice Exercises, Short Reading Practice, Review Sheet, Answer Key, Polish document overview, Polish key vocabulary, Polish topic

LLM-generated sections: Document Context Overview, Key Vocabulary, Important Verbs, Grammar Patterns, Useful Phrases and Expressions, Mini Language Lessons, Practice Exercises, Short Reading Practice, Review Sheet, Answer Key, Polish document overview, Polish key vocabulary, Polish topic vocabulary groups, Polish important verbs, Polish grammar patterns, Polish useful phrases, Polish mini lessons, Polish practice exercises, Polish reading practice, Polish review sheet, Polish answer key

section_name	provider	model	status	failed_attempts
Document Context Overview	groq	openai/gpt-oss-120b	Generated	
Key Vocabulary	groq	openai/gpt-oss-120b	Generated	
Important Verbs	groq	openai/gpt-oss-120b	Generated	
Grammar Patterns	groq	openai/gpt-oss-120b	Generated	
Useful Phrases and Expressions	groq	openai/gpt-oss-120b	Generated	
Mini Language Lessons	groq	openai/gpt-oss-120b	Generated	
Practice Exercises	groq	openai/gpt-oss-120b	Generated	
Short Reading Practice	groq	openai/gpt-oss-120b	Generated	
Review Sheet	groq	openai/gpt-oss-120b	Generated	
Answer Key	groq	openai/gpt-oss-120b	Generated	

Vocabulary

Verbs

Grammar

Exercises

171558

Output file: [test_doc_llm_learning_guide.pdf](#)

Download PDF Guide

Generated PDFs

name	size_kb	modified
test_doc_llm_learning_guide.pdf	46	2026-06-18 18:56:54

test_doc_llm_learning_guide.pdf46 KB

Download

Delete

This MVP flow outputs a static PDF learning guide.

See the generated document.

The followign images will showcase a sample generated pdf guide for the uploaded French document, you can see the table of contents, and some sectionnes of it. The pdf and the French document is available in the github repository:

Test Doc - French Learning Guide

French to English

Learner level: Intermediate

About This Guide

A static language-learning workbook generated from the uploaded document context.

Use the sections in order: overview, vocabulary, grammar, practice, review.

Table of Contents

1. Document Context Overview
2. Key Vocabulary
3. Topic-Based Vocabulary Groups
4. Important Verbs
5. Grammar Patterns
6. Useful Phrases and Expressions
7. Mini Language Lessons
8. Practice Exercises
9. Short Reading Practice
10. Review Sheet
11. Answer Key
12. Generation Information

4. Important Verbs

Verb	Meaning	Form / Note
comprendre	to understand	comprend; Elle comprend la situation – She understands the situation.
travailler	to work	travaille; Marie travaille dans les champs – Marie works in the fields.
organiser	to organize	organise; Elle organise des collectes de vivres – She organizes food collections.
décider	to decide	décide; Marie décide de quitter le village – Marie decides to leave the village.
découvrir	to discover	découvre; Elle découvre un environnement bruyant – She discovers a noisy environment.
apprendre	to learn	apprend; Elle apprend à manier la faux – She learns to handle the scythe.
aider	to help	aide; Elle aide les soldats blessés – She helps wounded soldiers.
rédiger	to write/draft	rédige; Marie rédige les courriers – Marie drafts the letters.
superviser	to supervise	supervise; Elle supervise la distribution des allocations – She supervises the distribution of allowances.
participer	to participate	participe; Elle participe aux discussions du conseil – She participates in council discussions.
changer	to change	change; Sa vie change radicalement – Her life changes radically.
devenir	to become	devient; Elle devient le moteur invisible – She becomes the invisible engine.
soutenir	to support	soutient; Elle soutient moralement les soldats – She morally supports the soldiers.
gérer	to manage	gère; Elle gère les stocks – She manages the stocks.
retourner	to return	retourne; Marie retourne dans son village – Marie returns to her village.

8. Practice Exercises

Exercise 1

Associez chaque mot français à sa traduction anglaise. (Match each French word with its English meaning.)

1. la blouse – le front – la tranchée –

Exercise 2

Complétez l'espace vide avec le mot approprié. (Fill in the blank with the correct word.)

1. Marie travaille ____ usine de munitions à Lyon.

Exercise 3

Choisissez l'article correct (le, la, l', un, une). (Select the correct article.)

1. ____ femme qui écrit des lettres est très patiente.

Exercise 4

Choisissez la forme verbale correcte. (Choose the correct verb form.)

1. En 1915, Marie ____ (décider) de quitter son village.

Exercise 5

Rédigez deux phrases décrivant le rôle de Marie en 1916. (Write two sentences describing Marie's role in 1916.)

1. Décrivez brièvement le travail de Marie en tant qu'infirmière bénévole en 1916.

Exercise 6

Reliez chaque expression à sa signification. (Match each expression with its meaning.)

1. prendre en charge – assurer la continuité – devenir le moteur –

Exercise 7

Complétez les dates manquantes. (Fill in the missing dates.)

1. Le conflit éclate en ____ (mois et année). Marie quitte son village au printemps ____ (année).

Exercise 8

Choisissez entre imparfait et passé composé. (Choose between imperfect and passé composé.)

1. Pendant la guerre, les femmes ____ (travailler) souvent douze heures par jour.

Annex C: Future improvements.

Known Limitations

- The MVP focuses on static PDF learning guides, not interactive exercises.
- WeasyPrint may require native system libraries on Windows. If those are unavailable, the application falls back to PyMuPDF for PDF generation.
- The current pipeline truncates LLM input to a maximum configured character length for the MVP.
- No database is implemented; storage is local file based.
- Target language only limited to French.

Future Improvements

- Solve the limitations above.
- Improve large-document handling by processing multiple chunks through the LLM layer.

Annex D: Acknowledgement.

We would like to acknowledge usage of generative AI in the process of development of this software. We hope that this product brings value to the users and be regarded as a useful tool for language learners.